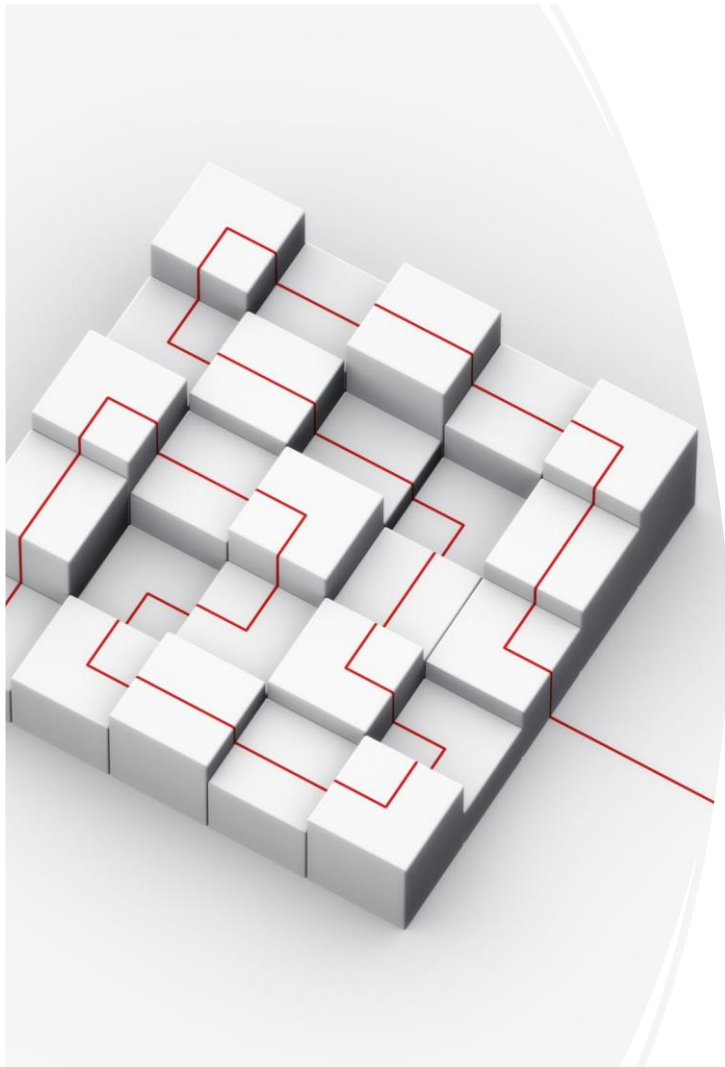


INT 161



Basic Backend Development
Week-04

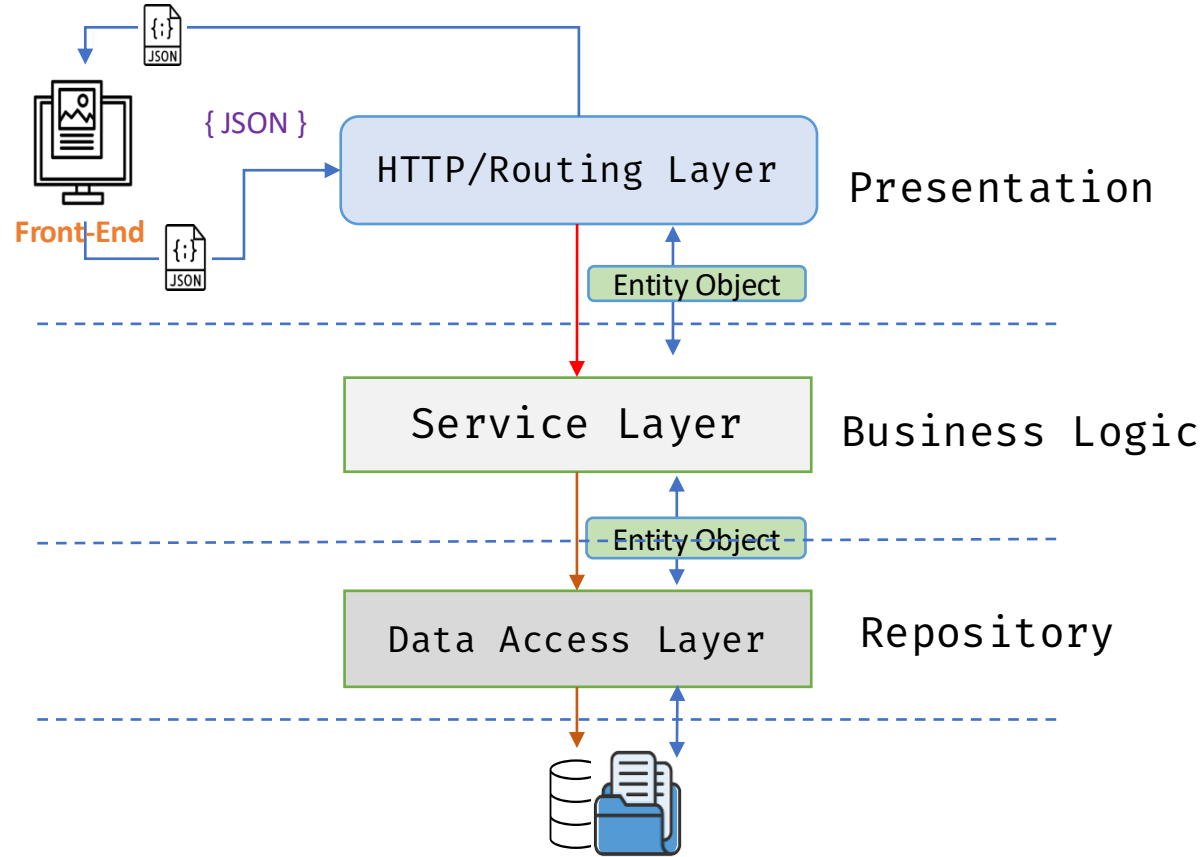
Database Connection



Unit Objectives

- After completing this unit, you should be able to:
 - Using Node.js connecting with MySQL
 - Create CRUD API
 - Using Prepared Statement to prevent SQL Injection
 - Understanding layer system code management

Layered System

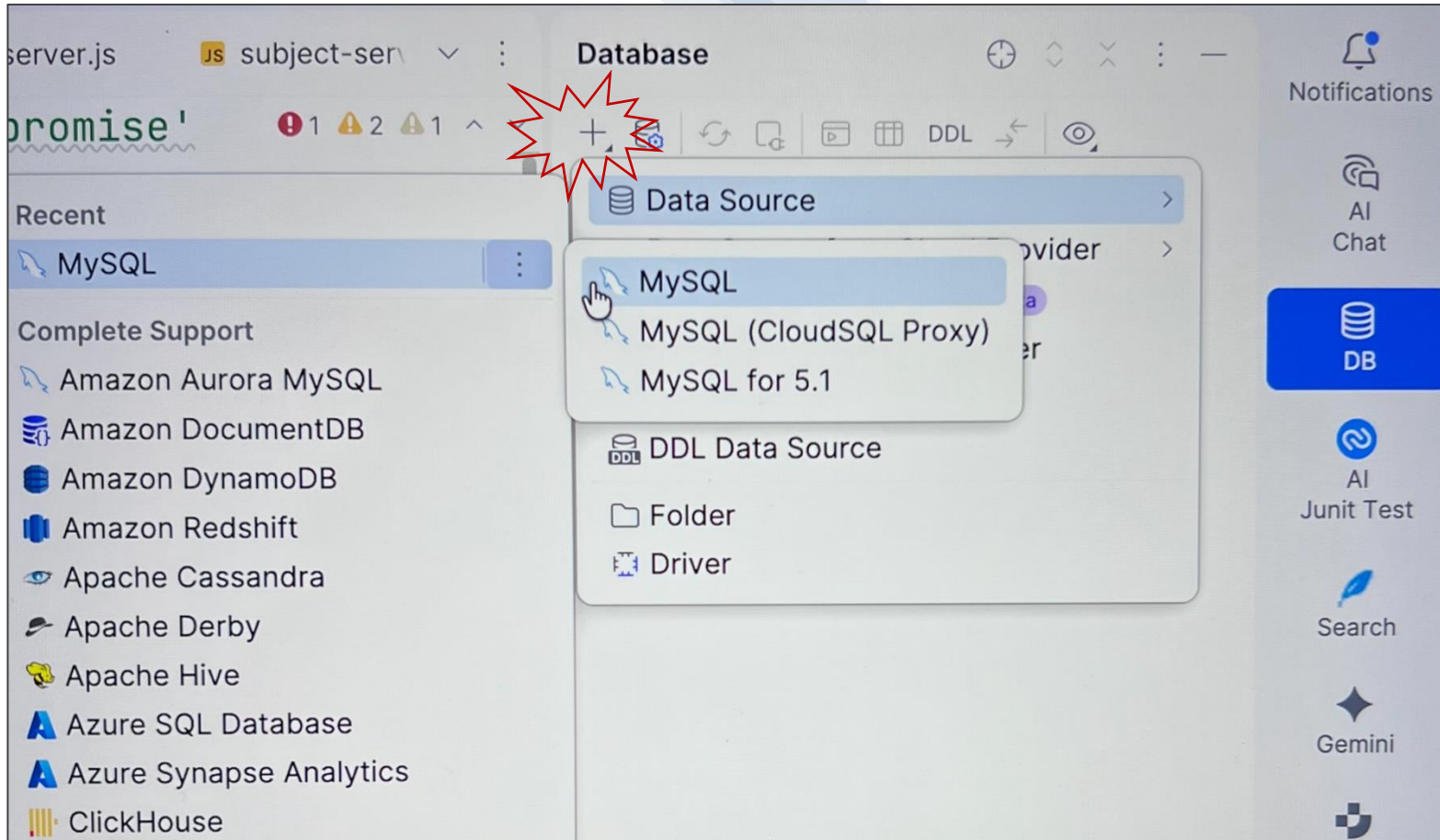


Concept: Hierarchy of intermediate layers where each layer only interacts with its adjacent neighbor

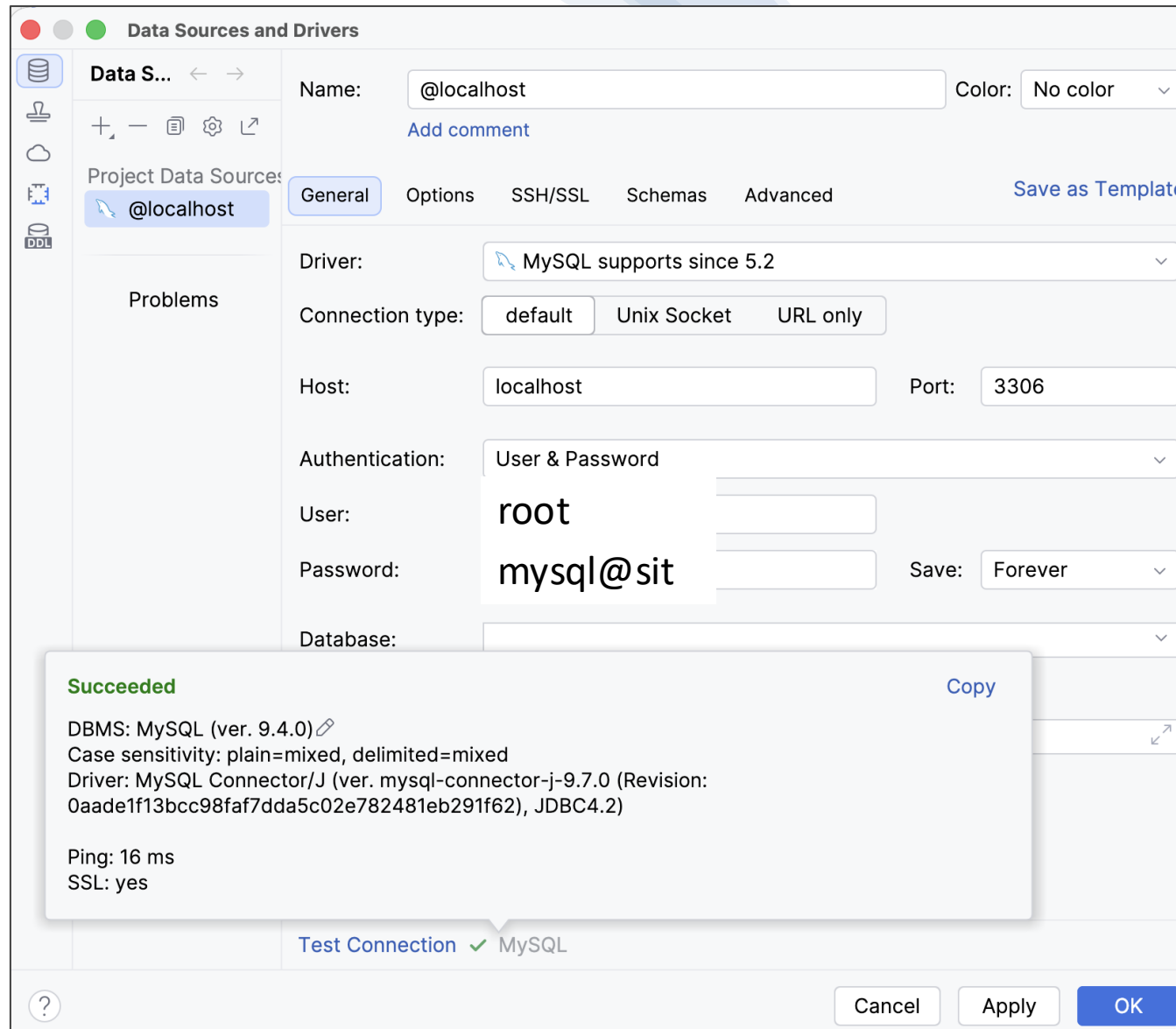
Benefits:

- **Scalability:** Shared caches and load balancers can be added without client changes
- **Separation of Concerns:** Keep raw HTTP parsing completely isolated from core business logic and database interactions
- **Security**

Using DB Client Tool in WebStorm: Create Data Source (1)



Using DB Client Tool in WebStorm: Create Data Source (2)



Using DB Client Tool: Create database sampled

```
console x
Tx: Auto Playground
1 ✓ create database `sampledb`;
2 ✓ use `sampledb`;
3 ✓ create table subjects
4 (
5     id          int primary key auto_increment,
6     subject_code varchar(8) not null unique,
7     subject_title varchar(60),
8     credit      int      not null
9 );
10
11 ✓ insert into subjects (subject_code, subject_title, credit)
12 values (subject_code 'INT 100', subject_title 'IT Fundamentals',
13         (subject_code 'INT 101', subject_title 'Programming Fundame
14         (subject_code 'INT 102', subject_title 'Web Technology', c
15         (subject_code 'INT 114', subject_title 'Discrete Mathematic
16         (subject_code 'GEN 101', subject_title 'Physical Education'
17         (subject_code 'GEN 111', subject_title 'Man and Ethics of L
18         (subject_code 'LNG 120', subject_title 'General English',
19         (subject_code 'LNG 220', subject_title 'Academic English',
20         (subject_code 'INT 103', subject_title 'Advanced Programmin
21         (subject_code 'INT 104', subject_title 'User Experience Des
22         (subject_code 'INT 105', subject_title 'Basic SQL', credit 1
23         (subject_code 'INT 107', subject_title 'Computing Platforms
24         (subject_code 'INT 200', subject_title 'Data Structures and
```

Create Table/Insert data

```
use sample;  
create table subjects  
(  
    id int primary key auto_increment,  
    subject_code varchar(8) not null unique,  
    subject_title varchar(60),  
    credit int not null  
);
```

```
insert into subjects (subject_code, subject_title, credit)  
values ('INT 100', 'IT Fundamentals', 3),  
      ('INT 101', 'Programming Fundamentals', 3),  
      ('INT 102', 'Web Technology', 1),  
      ('INT 114', 'Discrete Mathematics', 3),  
      ('GEN 101', 'Physical Education', 1),  
      ('GEN 111', 'Man and Ethics of Living', 3),  
      ('LNG 120', 'General English', 3),  
      ('LNG 220', 'Academic English', 3),  
      ('INT 103', 'Advanced Programming', 3),  
      ('INT 104', 'User Experience Design', 3),  
      ('INT 105', 'Basic SQL', 1),  
      ('INT 107', 'Computing Platforms Technology', 3),  
      ('INT 200', 'Data Structures and Algorithms', 1),  
      ('INT 201', 'Client-Side Programming I', 2),  
      ('INT 202', 'Server-Side Programming I', 2),  
      ('INT 205', 'Database Management System', 3),  
      ('INT 207', 'Network I', 3);
```

Step to connecting db in Node.js

1) Install Database Driver

```
> npm install mysql2
```

2) Create the Connection/Pool

```
import * as mysql from 'mysql2/promise'  
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: '143900',  
  database: 'sample'  
})
```

3) Send SQL command to DBMS

```
const results = await pool.query ("SQL Command")
```

<https://www.geeksforgeeks.org/node-js/node-js-connect-mysql-with-node-app/>

https://www.w3schools.com/nodejs/nodejs_mysql.asp

Mysql2 Library

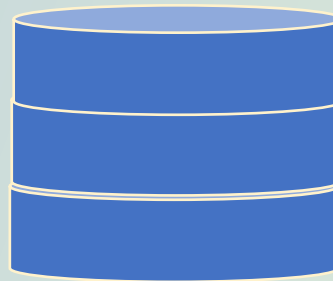
- Asynchronous (non-blocking I/O) using event loop
- Pure Node.js + native bindings using MySQL protocol
- Connection
 - Instance for connecting to DBMS
- Pool
 - Connection pool manager
- Query
 - Use to send SQL statement to DBMS
- Result
 - Array [rows, fields]
- Using
 - callback API (async style)
 - Promise API (async/await)

Send SQL to DBMS (Call Back vs Asyn/Await)

```
pool.query (  
  'SELECT * FROM students WHERE age > ?', [18], (err, results, fields) => {  
    if (err) {  
      console.error(' Query error:', err);  
      return;  
    }  
    console.log('Results:', results);  
  }  
);
```

```
async function main() {  
  const [rows] = await pool.query('SELECT * FROM students WHERE age > ?', [18]);  
  console.log(rows[0].message);  
}
```

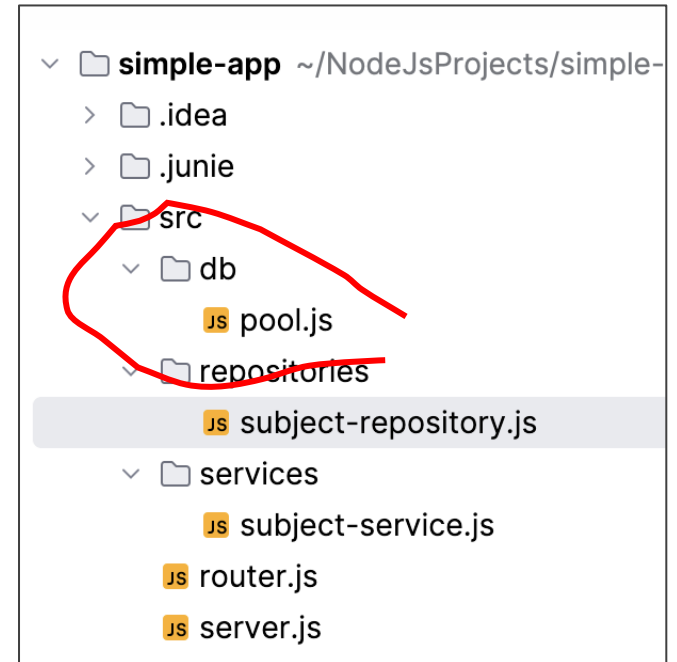
Test CRUD with Connection Pool



Create pool.js in folder ./db

```
import * as mysql from 'mysql2/promise'

export const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: '143900',
  database: 'sampledb'
})
export default pool;
```



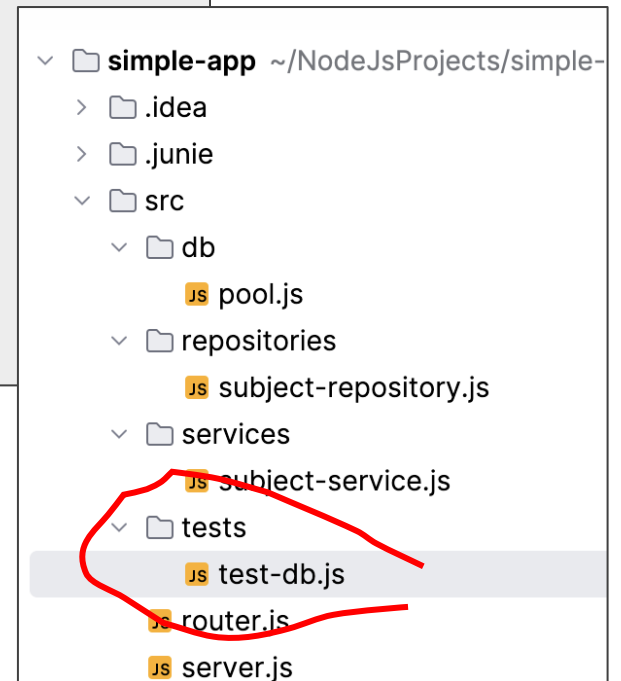
Create test-db in folder ./tests, then Run this file

```
import pool from "../db/pool.js"

export async function findAll() {
  const subjects = await pool.query ('SELECT * FROM subjects')
  return subjects;
}

const [subjects, table_schema] = await findAll();
await pool.end();

console.log(subjects);
console.log(table_schema);
```





Create REST API CRUD with Database

Modify subject-repository.js (1/3)

```
import pool from '../db/pool.js'

export async function findAll() {
  const [subjects] = await pool.query('SELECT * FROM subjects')
  return subjects;
}

export async function findById(id) {
  const [subject] = await pool.query(
    'SELECT * FROM subjects WHERE id = ?', [id])
  if (subject.length === 0) return null;
  return subject;
}

export async function remove(id) {
  const [result] = await pool.query(
    "DELETE FROM subjects WHERE id = ?", [id]);
  return result.affectedRows > 0;
}
```

Modify subject-repository.js (2/3)

```
export async function create(subject) {
  const [result] = await pool.query(
    "INSERT INTO subjects (subject_code, subject_title, credit) " +
    "VALUES (?, ?, ?)", [subject.code, subject.title, subject.credit]);
  console.log('Result: ', result);
  if (result.affectedRows === 0) return false;
  return findById(result.insertId);
}

export async function update(subject) {
  const [result] = await pool.query(
    "UPDATE subjects SET subject_code = ?, subject_title = ?, " +
    "credit = ? WHERE id = ?", [subject.code, subject.title,
    subject.credit, subject.id]
  );
  if (result.affectedRows === 0) return false;
  return findById(subject.id);
}
```

Modify router.js (1/5): Add function getBody

```
function getBody(request, response) {  
  return new Promise((resolve, reject) => {  
    let body = "";  
    // Collect data chunks as they arrive stream-by-stream  
    request.on('data', chunk => {  
      body += chunk.toString();  
    });  
    // Process the full payload once reception completes  
    request.on('end', () => {  
      // Parse the raw text buffer into a JavaScript object  
      if (body) {  
        const jsObject = JSON.parse(body);  
        console.log("object: ", jsObject);  
        resolve(jsObject);  
      } else {  
        resolve(null)  
      }  
    })  
  })  
}
```

```
});  
request.on('error', (err) => {  
  reject(err);  
});  
});  
}
```

Modify router.js (2/5): Add function writeResponse

```
async function writeResponse(response, status, message) {  
  response.writeHead(status, {'Content-Type': 'application/json'});  
  response.end(JSON.stringify(message));  
}
```

Modify router.js (3/5): update function createNewSubject

```
async function createNewSubject(request, response) {  
  const subject = await getBody(request, response);  
  var message = null  
  var status = 201  
  if (subject) {  
    // console.log(`Request body: ${JSON.stringify(subject)}`);  
    const newSubject = service.createNewSubject(subject)  
    message = newSubject  
    if (!newSubject) {  
      status = 409  
      message = {error: `Duplicate subject for id ${subject.id}`}  
    }  
  } else {  
    message = {error: "Bad request, body is empty"}  
    status = 400  
  }  
  writeResponse(response, status, message)  
}
```

Modify router.js (4/5): update function updateSubject

```
async function updateSubject(request, response, id) {  
  const subject = await getBody(request, response);  
  var message = null  
  var status = 200  
  if (subject) {  
    var newSubject = service.updateSubjectById(id, subject)  
    message = newSubject  
    console.log("update: ", message)  
    if (!newSubject) {  
      status = 404  
      message = {error: `Subject not found for id ${subject.id}`}  
    }  
  } else {  
    message = {error: "Bad request, body is empty"}  
    status = 400  
  }  
  writeResponse(response, status, message)  
}
```